Docker Infrastructure Setup Guide

MySQL, Redis & Apache Kafka — Complete Configuration Reference

Project: Real-Time Order Processing System **Version:** 2.0 | **Date:** March 2026 **Classification:** Internal — DevOps Reference

---

Table of Contents

1. Overview
2. Prerequisites
3. Architecture Diagram
4. Complete Docker Compose File
5. MySQL Setup
6. Redis Setup
7. Apache Kafka Setup
8. Service Dependencies & Boot Order
9. Networking
10. Volumes & Data Persistence
11. Environment Variables Reference
12. CLI Commands & Verification
13. Troubleshooting Guide
14. Production Enhancements

---

1. Overview

This document provides the complete Docker infrastructure setup for the Real-Time Order Processing System. The system uses three core infrastructure components:

Component	Image	Purpose	Port
MySQL 8.0	`mysql:8.0`	Relational database for 7 microservice databases	3307 → 3306
Redis 7	`redis:7-alpine`	Caching (Product Service), distributed locking (Inventory Service), rate limiting (API Gateway)	6379
Apache Kafka	`confluentinc/cp-kafka:7.5.0`	Event-driven async messaging between services (8 topics)	9092
ZooKeeper	`confluentinc/cp-zookeeper:7.5.0`	Kafka cluster coordination and metadata management	2181

Why Docker Compose?

```
WITHOUT DOCKER:
  1. Install MySQL → configure → create 7 databases → create user → grant privileges
  2. Install Redis → configure memory → set eviction policy
  3. Install ZooKeeper → configure → start
  4. Install Kafka → configure broker → configure listeners → start
```

```
  5. Install each microservice → configure connections → start in order
  → 2+ hours of manual setup, error-prone, non-reproducible

WITH DOCKER COMPOSE:
  docker compose up -d
  → 2 minutes. 100% reproducible. Works on any machine.
```

---

2. Prerequisites

Tool	Version	Purpose	Installation
Docker Desktop	4.0+	Container runtime	docker.com/download
Docker Compose	v2.0+	Multi-container orchestration	Included with Docker Desktop
Minimum RAM	8 GB	MySQL + Kafka + Redis + services	16 GB recommended
Disk Space	10 GB	Docker images + persistent volumes	

Verify Installation

```
# Check Docker version
docker --version
# Docker version 24.0.7, build afdd53b

# Check Docker Compose version
docker compose version
# Docker Compose version v2.23.0

# Check Docker is running
docker info
# Server Version: 24.0.7
```

---

3. Architecture Diagram

```
+──────────────────────────────────────────────────────────────────────────────+
|                    DOCKER COMPOSE NETWORK                                    |
|                (order-system-network: bridge)                                |
|                                                                              |
|  ┌──────────────────── INFRASTRUCTURE LAYER ──────────────────────────┐      |
|  |                                                                    |      |
|  |  ┌──────────────┐  ┌──────────────┐  ┌──────────┐  ┌──────────┐   |      |
|  |  |   MySQL 8.0  |  |  ZooKeeper   |  |  Kafka   |  |  Redis   |   |      |
|  |  |              |  |              |  |          |  |  7-alp   |   |      |
|  |  |  Port: 3307  |  |  Port: 2181  |  | Port:9092|  | Prt:6379|   |      |
|  |  |              |  |              |  |          |  |          |   |      |
|  |  |  7 databases |  |  Kafka coord |  | 8 topics |  | Cache +  |   |      |
|  |  |  auth_db     |  |              |  |          |  | Locks +  |   |      |
|  |  |  user_db     |  └──────┬───────┘  | Internal:|  | Rate     |   |      |
```

```
│  │  │  product_db  │         │          │ :29092   │  │ Limit   │  │  │
│  │  │  order_db    │         └──────────┴──────────┘  │         │  │  │
│  │  │  inventory_db│                    │          │  │         │  │  │
│  │  │  payment_db  │                    │          │  │         │  │  │
│  │  │  notif_db    │                    │          │  │         │  │  │
│  │  └──────┬───────┘                    └────┬─────┘  └────┬────┘  │  │
│  └─────────┼────────────────────────────────┼────────────┼───────┘  │
│            │                                │            │          │
│  ┌─────────┼──────── SERVICE LAYER ─────────┼────────────┼──────┐   │
│  │         │                                │            │      │   │
│  │         ▼             ▼                  ▼            ▼      │   │
│  │  ┌──────────┐  ┌─────────┐  ┌──────────────┐  ┌────────┐  │   │
│  │  │ Auth Svc │  │ User Svc │  │ Product Svc  │  │ Gateway │  │   │
│  │  │ (MySQL)  │  │ (MySQL)  │  │ (MySQL+Redis) │  │ (Redis) │  │   │
│  │  └──────────┘  └─────────┘  └──────────────┘  └────────┘  │   │
│  │                                                           │   │
│  │  ┌──────────┐  ┌──────────────┐  ┌──────────────────┐  │   │
│  │  │ Order Svc │  │ Inventory Svc  │  │ Payment Svc       │  │   │
│  │  │(MySQL+Kafka│  │(MySQL+Kafka+Redis│  │ (MySQL+Kafka)     │  │   │
│  │  └──────────┘  └──────────────┘  └──────────────────┘  │   │
│  │                                                           │   │
│  │  ┌──────────────────┐  ┌──────────────────────────────┐  │   │
│  │  │ Notification Svc  │  │ Frontend (Nginx :3000 → :80)  │  │   │
│  │  │ (MySQL+Kafka)     │  │                              │  │   │
│  │  └──────────────────┘  └──────────────────────────────┘  │   │
│  └──────────────────────────────────────────────────────────┘   │
+──────────────────────────────────────────────────────────────────+
```

---

4. Complete Docker Compose File

> ***File:*** *`docker-compose.yml` (project root)*

```yaml
services:
  # ==================== Infrastructure ====================

  # ── MySQL 8.0 ──────────────────────────────────────────
  # Relational database for all 7 microservices.
  # Each service has its own isolated database (Database-per-Service pattern).
  # The init.sql script auto-creates all databases and grants on first startup.
  mysql:
    image: mysql:8.0
    container_name: order-system-mysql
    environment:
      MYSQL_ROOT_PASSWORD: root123         # Root password (admin access)
      MYSQL_USER: orderuser                # Application user
      MYSQL_PASSWORD: orderpass            # Application password
    ports:
      - "3307:3306"                        # Host:3307 → Container:3306
                                           # (3307 avoids conflict with local MySQL)
    volumes:
      - mysql-data:/var/lib/mysql                          # Persistent data volume
```

```yaml
    - ./docker/mysql/init.sql:/docker-entrypoint-initdb.d/init.sql  # Auto-init script
  networks:
    - order-system-network
  healthcheck:
    test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
    interval: 10s                        # Check every 10 seconds
    timeout: 5s                          # Wait 5 seconds for response
    retries: 5                           # Fail after 5 consecutive failures

# ── ZooKeeper ──────────────────────────────────────────
# Kafka's coordination service. Manages broker metadata,
# leader election, and topic configuration.
# NOTE: Kafka 3.3+ supports KRaft mode (no ZooKeeper), but
# Confluent images still use ZooKeeper by default.
zookeeper:
  image: confluentinc/cp-zookeeper:7.5.0
  container_name: order-system-zookeeper
  environment:
    ZOOKEEPER_CLIENT_PORT: 2181          # Client connection port
    ZOOKEEPER_TICK_TIME: 2000            # Base time unit (ms) for heartbeats
  ports:
    - "2181:2181"
  networks:
    - order-system-network

# ── Apache Kafka ───────────────────────────────────────
# Distributed event streaming platform.
# Handles async communication between all microservices.
# Topics: order-created, order-cancelled, inventory-updated,
#         payment-success, payment-failure, product-created,
#         product-events, audit-logs
kafka:
  image: confluentinc/cp-kafka:7.5.0
  container_name: order-system-kafka
  depends_on:
    - zookeeper                          # Kafka needs ZooKeeper running first
  ports:
    - "9092:9092"                        # External access (host machine)
  environment:
    KAFKA_BROKER_ID: 1                   # Unique broker identifier
    KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181  # ZooKeeper address (Docker DNS)

    # ── Listener Configuration ──────────────────────────
    # PLAINTEXT (kafka:29092) → Internal Docker network access
    #   Used by: order-service, inventory-service, payment-service, etc.
    #   These services connect via Docker DNS name "kafka"
    #
    # PLAINTEXT_HOST (localhost:9092) → External host machine access
    #   Used by: developers running services outside Docker (IDE debugging)
    KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:29092,PLAINTEXT_HOST://localhost:9092
    KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: PLAINTEXT:PLAINTEXT,PLAINTEXT_HOST:PLAINTEXT
    KAFKA_INTER_BROKER_LISTENER_NAME: PLAINTEXT
```

```yaml
    KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1    # Single broker → replication=1
    KAFKA_AUTO_CREATE_TOPICS_ENABLE: "true"      # Topics auto-created on first publish
  networks:
    - order-system-network

# ── Redis 7 ──────────────────────────────────────────────
# In-memory data store used for:
#   1. Product caching (@Cacheable) in Product Service
#   2. Distributed locking (SETNX) in Inventory Service
#   3. Rate limiting backend in API Gateway
redis:
  image: redis:7-alpine                  # Alpine = smaller image (~5MB base)
  container_name: order-system-redis
  ports:
    - "6379:6379"                        # Default Redis port
  volumes:
    - redis-data:/data                   # Persistent volume for RDB/AOF snapshots
  networks:
    - order-system-network

# ==================== Core Services ====================

eureka-server:
  build: ./backend/eureka-server
  container_name: order-system-eureka
  ports:
    - "8761:8761"
  networks:
    - order-system-network
  healthcheck:
    test: ["CMD-SHELL", "curl -f http://localhost:8761/actuator/health || exit 1"]
    interval: 30s
    timeout: 10s
    retries: 3

api-gateway:
  build:
    context: .
    dockerfile: backend/api-gateway/Dockerfile
  container_name: order-system-gateway
  ports:
    - "8088:8080"                        # External: 8088, Internal: 8080
  environment:
    EUREKA_CLIENT_SERVICEURL_DEFAULTZONE: http://eureka-server:8761/eureka/
    SPRING_DATA_REDIS_HOST: redis        # Redis for rate limiting
  depends_on:
    eureka-server:
      condition: service_healthy
    redis:
      condition: service_started
  networks:
```

```yaml
      - order-system-network

  # ==================== Microservices ====================

  auth-service:
    build: ./backend/auth-service
    container_name: order-system-auth
    environment:
      SPRING_DATASOURCE_URL: jdbc:mysql://mysql:3306/auth_db?
useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC
      EUREKA_CLIENT_SERVICEURL_DEFAULTZONE: http://eureka-server:8761/eureka/
    depends_on:
      mysql:
        condition: service_healthy
      eureka-server:
        condition: service_healthy
    networks:
      - order-system-network

  user-service:
    build: ./backend/user-service
    container_name: order-system-user
    environment:
      SPRING_DATASOURCE_URL: jdbc:mysql://mysql:3306/user_db?
useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC
      EUREKA_CLIENT_SERVICEURL_DEFAULTZONE: http://eureka-server:8761/eureka/
    depends_on:
      mysql:
        condition: service_healthy
      eureka-server:
        condition: service_healthy
    networks:
      - order-system-network

  product-service:
    build: ./backend/product-service
    container_name: order-system-product
    environment:
      SPRING_DATASOURCE_URL: jdbc:mysql://mysql:3306/product_db?
useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC
      EUREKA_CLIENT_SERVICEURL_DEFAULTZONE: http://eureka-server:8761/eureka/
      SPRING_DATA_REDIS_HOST: redis        # Redis for product caching
    depends_on:
      mysql:
        condition: service_healthy
      eureka-server:
        condition: service_healthy
      redis:
        condition: service_started
    networks:
      - order-system-network
```

```yaml
  order-service:
    build: ./backend/order-service
    container_name: order-system-order
    environment:
      SPRING_DATASOURCE_URL: jdbc:mysql://mysql:3306/order_db?
useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC
      EUREKA_CLIENT_SERVICEURL_DEFAULTZONE: http://eureka-server:8761/eureka/
      SPRING_KAFKA_BOOTSTRAP_SERVERS: kafka:29092   # Kafka internal listener
    depends_on:
      mysql:
        condition: service_healthy
      eureka-server:
        condition: service_healthy
      kafka:
        condition: service_started
    networks:
      - order-system-network

  inventory-service:
    build:
      context: .
      dockerfile: backend/inventory-service/Dockerfile
    container_name: order-system-inventory
    environment:
      SPRING_DATASOURCE_URL: jdbc:mysql://mysql:3306/inventory_db?
useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC
      EUREKA_CLIENT_SERVICEURL_DEFAULTZONE: http://eureka-server:8761/eureka/
      SPRING_KAFKA_BOOTSTRAP_SERVERS: kafka:29092   # Kafka for event consumption
      SPRING_DATA_REDIS_HOST: redis                 # Redis for distributed locking
    depends_on:
      mysql:
        condition: service_healthy
      eureka-server:
        condition: service_healthy
      kafka:
        condition: service_started
      redis:
        condition: service_started
    networks:
      - order-system-network

  payment-service:
    build:
      context: .
      dockerfile: backend/payment-service/Dockerfile
    container_name: order-system-payment
    environment:
      SPRING_DATASOURCE_URL: jdbc:mysql://mysql:3306/payment_db?
useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC
      EUREKA_CLIENT_SERVICEURL_DEFAULTZONE: http://eureka-server:8761/eureka/
      SPRING_KAFKA_BOOTSTRAP_SERVERS: kafka:29092
    depends_on:
```

```yaml
      mysql:
        condition: service_healthy
      eureka-server:
        condition: service_healthy
      kafka:
        condition: service_started
    networks:
      - order-system-network

  notification-service:
    build: ./backend/notification-service
    container_name: order-system-notification
    environment:
      SPRING_DATASOURCE_URL: jdbc:mysql://mysql:3306/notification_db?
useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC
      EUREKA_CLIENT_SERVICEURL_DEFAULTZONE: http://eureka-server:8761/eureka/
      SPRING_KAFKA_BOOTSTRAP_SERVERS: kafka:29092
    depends_on:
      mysql:
        condition: service_healthy
      eureka-server:
        condition: service_healthy
      kafka:
        condition: service_started
    networks:
      - order-system-network

  # ==================== Frontend ====================

  frontend:
    build: ./frontend
    container_name: order-system-frontend
    ports:
      - "3000:80"                          # Nginx serves on port 80 internally
    depends_on:
      api-gateway:
        condition: service_started
    networks:
      - order-system-network

# ==================== Volumes ====================
volumes:
  mysql-data:                              # Persistent MySQL data
  redis-data:                              # Persistent Redis snapshots

# ==================== Network ====================
networks:
  order-system-network:
    driver: bridge                         # Default Docker bridge network
```

5. MySQL Setup

5.1 Image & Configuration

```yaml
mysql:
  image: mysql:8.0                        # MySQL 8.0 (LTS)
  container_name: order-system-mysql
  environment:
    MYSQL_ROOT_PASSWORD: root123           # Root admin password
    MYSQL_USER: orderuser                 # Application-level user
    MYSQL_PASSWORD: orderpass             # Application-level password
  ports:
    - "3307:3306"                         # Mapped to 3307 to avoid local MySQL conflict
```

WHY port 3307? Most developers have MySQL installed locally on port 3306. Using 3307 for Docker ensures no conflict. Services inside Docker still connect on port 3306 (via Docker DNS: `mysql:3306`).

5.2 Database Initialization Script

> ***File:*** *`docker/mysql/init.sql`*

```sql
-- Create databases for each microservice (Database-per-Service pattern)
CREATE DATABASE IF NOT EXISTS auth_db;
CREATE DATABASE IF NOT EXISTS user_db;
CREATE DATABASE IF NOT EXISTS product_db;
CREATE DATABASE IF NOT EXISTS order_db;
CREATE DATABASE IF NOT EXISTS inventory_db;
CREATE DATABASE IF NOT EXISTS payment_db;
CREATE DATABASE IF NOT EXISTS notification_db;

-- Grant privileges to the application user
GRANT ALL PRIVILEGES ON auth_db.* TO 'orderuser'@'%';
GRANT ALL PRIVILEGES ON user_db.* TO 'orderuser'@'%';
GRANT ALL PRIVILEGES ON product_db.* TO 'orderuser'@'%';
GRANT ALL PRIVILEGES ON order_db.* TO 'orderuser'@'%';
GRANT ALL PRIVILEGES ON inventory_db.* TO 'orderuser'@'%';
GRANT ALL PRIVILEGES ON payment_db.* TO 'orderuser'@'%';
GRANT ALL PRIVILEGES ON notification_db.* TO 'orderuser'@'%';

FLUSH PRIVILEGES;
```

HOW IT WORKS:

- `docker-entrypoint-initdb.d/` is a special MySQL Docker directory
- Any `.sql` file placed here runs **only on first startup** (empty volume)
- Subsequent `docker compose up` calls skip this script (data persists)

5.3 Health Check Explained

```yaml
healthcheck:
  test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
  interval: 10s      # Check every 10 seconds
  timeout: 5s        # Consider failed if no response in 5 seconds
  retries: 5         # Mark unhealthy after 5 consecutive failures
```

WHY health checks matter:

- Services that `depends_on` MySQL with `condition: service_healthy` wait until MySQL is READY
- Without healthcheck: service starts when MySQL container starts (NOT when MySQL is accepting connections)
- With healthcheck: service starts only after MySQL responds to `mysqladmin ping` → no `Connection refused` errors

5.4 Database-per-Service Pattern

```
+-------------------+-------------------+-------------------------------+
| Service           | Database          | Tables (auto-created by JPA) |
+-------------------+-------------------+-------------------------------+
| Auth Service      | auth_db           | users, roles                  |
| User Service      | user_db           | users, addresses              |
| Product Service   | product_db        | products                      |
| Order Service     | order_db          | orders, order_items           |
| Inventory Service| inventory_db      | inventory, processed_events   |
| Payment Service   | payment_db        | payments                      |
| Notification Svc  | notification_db   | notifications, audit_logs     |
+-------------------+-------------------+-------------------------------+

WHY separate databases?
  1. Isolation: One service's schema change can't break another
  2. Scalability: Each database can be moved to its own server
  3. Security: Each service only has access to its own data
  4. Independence: Teams can evolve schemas independently
```

5.5 Connecting from Spring Boot

```yaml
# Inside Docker (services use Docker DNS):
SPRING_DATASOURCE_URL: jdbc:mysql://mysql:3306/order_db?
useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC

# Outside Docker (local IDE development):
spring:
  datasource:
    url: jdbc:mysql://localhost:3307/order_db?
useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC
    username: orderuser
    password: orderpass
```

5.6 MySQL CLI Access

```
# Connect to MySQL container
docker exec -it order-system-mysql mysql -u orderuser -porderpass

# List all databases
mysql> SHOW DATABASES;
+--------------------+
| Database           |
+--------------------+
| auth_db            |
| user_db            |
| product_db         |
| order_db           |
| inventory_db       |
| payment_db         |
| notification_db    |
+--------------------+

# Use a specific database
mysql> USE order_db;
mysql> SHOW TABLES;
mysql> SELECT * FROM orders LIMIT 5;

# Check active connections
mysql> SHOW PROCESSLIST;
```

---

6. Redis Setup

6.1 Image & Configuration

```yaml
redis:
  image: redis:7-alpine                   # Redis 7.x on Alpine Linux
  container_name: order-system-redis
  ports:
    - "6379:6379"                          # Default Redis port
  volumes:
    - redis-data:/data                     # Persistent volume
```

WHY `redis:7-alpine`?

- Alpine: ~5MB base image vs ~100MB Debian → faster pulls, smaller footprint
- Redis 7: Latest stable with ACL improvements, multi-part AOF, function support
- No custom `redis.conf` needed for development (defaults are sufficient)

6.2 Redis Usage in This Project

```
+-------------------+-------------------+-------------------+-------------------+
| Service           | Purpose           | Redis Commands    | Key Pattern       |
+-------------------+-------------------+-------------------+-------------------+
| Product Service   | Cache-Aside       | GET, SET, DEL     | product::1        |
```

```
|                  | caching          | (via Spring)     | products::all    |
|                  |                  | TTL: 10min/5min  |                  |
+------------------+------------------+------------------+------------------+
| Inventory Service| Distributed      | SET NX EX 30     | inventory:lock:  |
|                  | locking          | GET, DEL         | {productId}      |
|                  |                  | TTL: 30 seconds  |                  |
+------------------+------------------+------------------+------------------+
| API Gateway      | Rate limiting    | INCR, EXPIRE     | (internal)       |
|                  |                  | TTL: 60 seconds  |                  |
+------------------+------------------+------------------+------------------+
```

6.3 Connecting from Spring Boot

```yaml
# Inside Docker:
SPRING_DATA_REDIS_HOST: redis    # Docker DNS resolves to Redis container IP

# Outside Docker (local IDE):
spring:
  data:
    redis:
      host: localhost
      port: 6379
```

6.4 Redis CLI Access

```bash
# Connect to Redis container
docker exec -it order-system-redis redis-cli

# Test connectivity
127.0.0.1:6379> PING
PONG

# View all cached keys
127.0.0.1:6379> KEYS *
1) "product::1"
2) "products::SimpleKey []"
3) "inventory:lock:42"

# Check TTL of a cached product
127.0.0.1:6379> TTL product::1
(integer) 542     # 542 seconds remaining

# View cached product data (JSON)
127.0.0.1:6379> GET product::1
"{\"@class\":\"...ProductDto\",\"id\":1,\"name\":\"Laptop\",\"price\":999.99}"

# Check memory usage
127.0.0.1:6379> INFO memory
used_memory_human:1.24M
```

```
# Monitor commands in real-time (debugging)
127.0.0.1:6379> MONITOR
OK
1614695423.456 [0 172.18.0.5:38012] "GET" "product::1"
```

6.5 Production Redis Configuration

For production, add a custom `redis.conf` :

```yaml
redis:
  image: redis:7-alpine
  container_name: order-system-redis
  command: redis-server /usr/local/etc/redis/redis.conf
  volumes:
    - redis-data:/data
    - ./docker/redis/redis.conf:/usr/local/etc/redis/redis.conf
```

```
# docker/redis/redis.conf
maxmemory 256mb
maxmemory-policy allkeys-lru
requirepass ${REDIS_PASSWORD}
appendonly yes
appendfsync everysec
save 300 100
```

---

7. Apache Kafka Setup

7.1 ZooKeeper Configuration

```yaml
zookeeper:
  image: confluentinc/cp-zookeeper:7.5.0   # Confluent Platform (production-grade)
  container_name: order-system-zookeeper
  environment:
    ZOOKEEPER_CLIENT_PORT: 2181            # Port for Kafka to connect
    ZOOKEEPER_TICK_TIME: 2000              # Base heartbeat unit (2 seconds)
```

WHY ZooKeeper? ZooKeeper manages Kafka's cluster metadata: broker registration, topic configurations, partition leader election, and consumer group coordination.

7.2 Kafka Broker Configuration

```yaml
kafka:
  image: confluentinc/cp-kafka:7.5.0
  container_name: order-system-kafka
  depends_on:
    - zookeeper
  ports:
    - "9092:9092"
```

```yaml
    environment:
      KAFKA_BROKER_ID: 1
      KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:29092,PLAINTEXT_HOST://localhost:9092
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: PLAINTEXT:PLAINTEXT,PLAINTEXT_HOST:PLAINTEXT
      KAFKA_INTER_BROKER_LISTENER_NAME: PLAINTEXT
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
      KAFKA_AUTO_CREATE_TOPICS_ENABLE: "true"
```

7.3 Listener Configuration Deep Dive

```
KAFKA DUAL-LISTENER ARCHITECTURE:

  ┌──────────────────────────────────────────────────────────────┐
  │                    KAFKA BROKER                              │
  │                                                              │
  │  LISTENER 1: PLAINTEXT (kafka:29092)                         │
  │  ├─ WHO USES IT: Docker services (order-service, etc.)       │
  │  ├─ HOW: Via Docker DNS → "kafka" resolves to container IP   │
  │  └─ SPRING CONFIG: SPRING_KAFKA_BOOTSTRAP_SERVERS=kafka:29092 │
  │                                                              │
  │  LISTENER 2: PLAINTEXT_HOST (localhost:9092)                 │
  │  ├─ WHO USES IT: Developer's local IDE (IntelliJ, VS Code)   │
  │  ├─ HOW: Port mapping 9092:9092 → localhost:9092             │
  │  └─ SPRING CONFIG: bootstrap-servers: localhost:9092          │
  └──────────────────────────────────────────────────────────────┘

  WHY TWO LISTENERS?
  Without PLAINTEXT_HOST: Services running OUTSIDE Docker cannot reach Kafka.
    → localhost:9092 doesn't work because Kafka advertises "kafka:29092"
    → The host machine can't resolve "kafka" DNS → connection fails
  With PLAINTEXT_HOST: Kafka advertises "localhost:9092" for host connections.
```

7.4 Topic Configuration

```
TOPICS IN THIS PROJECT (auto-created):

+------------------+------------------+------------------+------------------+
| Topic            | Producer         | Consumer(s)      | Partition Key    |
+------------------+------------------+------------------+------------------+
| product-created  | Product Service  | Inventory Service| productId        |
| product-events   | Product Service  | (analytics)      | productId        |
| order-created    | Order Service    | Inventory Service| productId        |
| order-cancelled  | Order Service    | Inventory Service| productId        |
| inventory-updated| Inventory Svc    | Order, Payment   | orderId          |
| payment-success  | Payment Service  | Order, Notif.    | orderId          |
| payment-failure  | Payment Service  | Order, Notif.    | orderId          |
| audit-logs       | All services     | Notification Svc | —                |
+------------------+------------------+------------------+------------------+
```

7.5 Connecting from Spring Boot

```yaml
# Inside Docker:
SPRING_KAFKA_BOOTSTRAP_SERVERS: kafka:29092

# Outside Docker (local IDE):
spring:
  kafka:
    bootstrap-servers: localhost:9092
```

7.6 Kafka CLI Access

```bash
# List all topics
docker exec -it order-system-kafka kafka-topics \
  --bootstrap-server localhost:29092 --list

# Describe a topic
docker exec -it order-system-kafka kafka-topics \
  --bootstrap-server localhost:29092 \
  --describe --topic order-created

# Consume messages from a topic (real-time)
docker exec -it order-system-kafka kafka-console-consumer \
  --bootstrap-server localhost:29092 \
  --topic order-created --from-beginning

# Produce a test message
docker exec -it order-system-kafka kafka-console-producer \
  --bootstrap-server localhost:29092 \
  --topic order-created

# Check consumer group lag
docker exec -it order-system-kafka kafka-consumer-groups \
  --bootstrap-server localhost:29092 \
  --describe --group inventory-service-group
```

---

8. Service Dependencies & Boot Order

```
BOOT ORDER (enforced by depends_on):

  Phase 1: Infrastructure (no dependencies)
  ┌──────────┐   ┌──────────┐   ┌──────────┐
  │  MySQL   │   │ ZooKeeper│   │  Redis   │
  └──────┬───┘   └──────┬───┘   └──────┬───┘
         │              │              │
         │              ▼              │
         │         ┌──────────┐        │
         │         │  Kafka   │        │
```

```
        │         └──────┬──────┘          │
        │                │                 │
  Phase 2: Core Services (need infrastructure healthy)
        ▼                │                 │
  ┌──────────┐           │                 │
  │  Eureka  │           │                 │
  └──────┬───┘           │                 │
        │                │                 │
  Phase 3: Application Services
        ▼                ▼                 ▼
  ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐
  │   Auth   │  │  Order   │  │ Inventory│  │ Product  │
  └──────────┘  └──────────┘  └──────────┘  └──────────┘

  ┌──────────┐  ┌──────────┐  ┌──────────┐
  │   User   │  │ Payment  │  │  Notif.  │
  └──────────┘  └──────────┘  └──────────┘


  Phase 4: Frontend

  ┌──────────┐
  │ Frontend │ (waits for Gateway)
  └──────────┘


depends_on conditions:
  service_healthy  → wait until healthcheck passes (MySQL, Eureka)
  service_started  → wait until container starts (Kafka, Redis)
```

---

9. Networking

```yaml
networks:
  order-system-network:
    driver: bridge         # Default Docker bridge network
```

HOW DOCKER DNS WORKS:

```
Every container on the same network can reach others by container name:
  mysql           → 172.18.0.2:3306
  redis           → 172.18.0.3:6379
  kafka           → 172.18.0.4:29092
  zookeeper       → 172.18.0.5:2181
  eureka-server   → 172.18.0.6:8761

Spring Boot services use these DNS names in environment variables:
  SPRING_DATASOURCE_URL=jdbc:mysql://mysql:3306/order_db
  SPRING_DATA_REDIS_HOST=redis
  SPRING_KAFKA_BOOTSTRAP_SERVERS=kafka:29092
  EUREKA_CLIENT_SERVICEURL_DEFAULTZONE=http://eureka-server:8761/eureka/
```

---

10. Volumes & Data Persistence

```yaml
volumes:
  mysql-data:      # Persists MySQL databases between restarts
  redis-data:      # Persists Redis RDB/AOF snapshots between restarts
```

```bash
# View Docker volumes
docker volume ls
# DRIVER    VOLUME NAME
# local     real-time-order-processing-system_mysql-data
# local     real-time-order-processing-system_redis-data

# Inspect a volume (find physical location)
docker volume inspect real-time-order-processing-system_mysql-data

# ⚠️ Delete all data (fresh start)
docker compose down -v                    # -v removes volumes too
```

IMPORTANT: `docker compose down` preserves volumes. `docker compose down -v` DELETES volumes (all data lost).

---

11. Environment Variables Reference

Variable	Service(s)	Value (Docker)	Value (Local)
`SPRING_DATASOURCE_URL`	All (except Gateway)	`jdbc:mysql://mysql:3306/{db}`	`jdbc:mysql://localhost:3`
`SPRING_DATA_REDIS_HOST`	Product, Inventory, Gateway	`redis`	`localhost`
`SPRING_KAFKA_BOOTSTRAP_SERVERS`	Order, Inventory, Payment, Notification	`kafka:29092`	`localhost:9092`
`EUREKA_CLIENT_SERVICEURL_DEFAULTZONE`	All services	`http://eureka-server:8761/eureka/`	`http://localhost:8761/eu`
`MYSQL_ROOT_PASSWORD`	MySQL	`root123`	—
`MYSQL_USER`	MySQL	`orderuser`	—
`MYSQL_PASSWORD`	MySQL	`orderpass`	—

---

12. CLI Commands & Verification

12.1 Docker Compose Commands

```
# Start all services (detached mode)
docker compose up -d

# Start only infrastructure
docker compose up -d mysql redis zookeeper kafka

# View running containers
docker compose ps

# View logs for a specific service
docker compose logs -f order-service

# Stop all services (keep data)
docker compose down

# Stop all services (DELETE all data)
docker compose down -v

# Rebuild a specific service
docker compose build order-service
docker compose up -d order-service

# Restart a specific service
docker compose restart inventory-service

# Scale a service (multiple instances)
docker compose up -d --scale order-service=3
```

12.2 Verification Checklist

```
# 1. Check all containers are running
docker compose ps
# Expected: all services show "Up" or "Up (healthy)"

# 2. Verify MySQL is accepting connections
docker exec -it order-system-mysql mysqladmin ping -u orderuser -porderpass
# Expected: mysqld is alive

# 3. Verify Redis is responding
docker exec -it order-system-redis redis-cli PING
# Expected: PONG

# 4. Verify Kafka broker is ready
docker exec -it order-system-kafka kafka-topics --bootstrap-server localhost:29092 --list
# Expected: list of topics (may be empty initially)

# 5. Verify Eureka has registered services
curl http://localhost:8761/eureka/apps
# Expected: XML with registered service instances
```

```
# 6. Test API Gateway
curl http://localhost:8088/api/auth/register
# Expected: 4xx or response (not connection refused)

# 7. Check service logs for errors
docker compose logs --tail=50 order-service | grep ERROR
```

---

13. Troubleshooting Guide

13.1 Common Issues

Problem	Cause	Solution
`Connection refused` to MySQL	MySQL not ready yet	Use `depends_on: condition: service_healthy`
`Connection refused` to Kafka	Kafka not fully started	Wait 30-60s after compose up OR add healthcheck
`Unknown host: mysql`	Container not on same network	Verify `order-system-network` in all services
`Port already in use: 3306`	Local MySQL running	Stop local MySQL OR change port to 3307 (done)
`Port already in use: 6379`	Local Redis running	Stop local Redis: `redis-cli shutdown`
`Out of memory`	Not enough RAM	Increase Docker Desktop memory to 8GB+
`Kafka: Leader not available`	Topic not yet created	Wait for auto-creation on first publish
`Redis: READONLY`	Connected to replica	Ensure connecting to master node
Services can't find Eureka	Eureka not healthy yet	Increase healthcheck timeout

13.2 Debug Commands

```
# Check container resource usage
docker stats

# Inspect container network
docker inspect order-system-mysql | grep -A 10 "Networks"

# View container environment variables
docker inspect order-system-order | grep -A 50 "Env"

# Access container shell
docker exec -it order-system-order /bin/sh

# Check Docker network
docker network inspect real-time-order-processing-system_order-system-network
```

```
# Force recreate all containers
docker compose up -d --force-recreate
```

---

14. Production Enhancements

14.1 Security Hardening

```yaml
# Production docker-compose.override.yml
services:
  mysql:
    environment:
      MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}    # From .env file
      MYSQL_USER: ${MYSQL_USER}
      MYSQL_PASSWORD: ${MYSQL_PASSWORD}

  redis:
    command: redis-server --requirepass ${REDIS_PASSWORD} --maxmemory 256mb --maxmemory-
policy allkeys-lru

  kafka:
    environment:
      KAFKA_OPTS: "-Djava.security.auth.login.config=/etc/kafka/kafka_jaas.conf"
```

14.2 Resource Limits

```yaml
services:
  mysql:
    deploy:
      resources:
        limits:
          cpus: '2.0'
          memory: 2G
        reservations:
          cpus: '1.0'
          memory: 1G

  kafka:
    deploy:
      resources:
        limits:
          cpus: '2.0'
          memory: 2G

  redis:
    deploy:
      resources:
        limits:
```

```
          cpus: '1.0'
          memory: 512M
```

14.3 Production Kafka (Multi-Broker)

```
kafka-1:
  image: confluentinc/cp-kafka:7.5.0
  environment:
    KAFKA_BROKER_ID: 1
    KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 3
    KAFKA_DEFAULT_REPLICATION_FACTOR: 3
    KAFKA_MIN_INSYNC_REPLICAS: 2

kafka-2:
  image: confluentinc/cp-kafka:7.5.0
  environment:
    KAFKA_BROKER_ID: 2
    # ... same config, different broker ID

kafka-3:
  image: confluentinc/cp-kafka:7.5.0
  environment:
    KAFKA_BROKER_ID: 3
    # ... same config, different broker ID
```

14.4 Monitoring Stack

```
# Add monitoring to docker-compose
prometheus:
  image: prom/prometheus:latest
  ports:
    - "9090:9090"
  volumes:
    - ./docker/prometheus/prometheus.yml:/etc/prometheus/prometheus.yml

grafana:
  image: grafana/grafana:latest
  ports:
    - "3001:3000"
  environment:
    GF_SECURITY_ADMIN_PASSWORD: admin

redis-exporter:
  image: oliver006/redis_exporter:latest
  environment:
    REDIS_ADDR: redis://redis:6379

kafka-exporter:
```

```
    image: danielqsj/kafka-exporter:latest
    command: --kafka.server=kafka:29092
```

End of Document

Run `docker compose up -d` from the project root to start the entire infrastructure. Use the verification checklist (Section 12.2) to confirm all services are healthy. For local IDE development, use the "Outside Docker" connection strings from Section 11.